

Semana 1, sesión 1

1. Presentación del curso

Computación científica, aplicaciones en Ciencia de Datos,
Machine Learning y Optimización

Métodos Numéricos

Edwin Fernando Muñoz

Contenido

Presentación del curso

- Información general

- Presentación

- Competencias y resultados de aprendizaje

- Contenidos

- Metodología

- Evaluación

Computación científica y aplicaciones

- Motivación

- Modelación matemática

- Computación científica aplicada

- Python para computación científica

- Soluciones computacionales y errores

- Aplicaciones en Optimización

- ¿Por qué Métodos Numéricos en Ciencia de Datos?

Información general

Programa	Ciencia de Datos
Área	Matemáticas
Asignatura	Métodos Numéricos
Créditos	4
Semestre	Quinto
Horas presenciales	64 horas semestrales
Trabajo autónomo	64 horas semestrales
Profesor	Edwin Fernando Muñoz Chipatecua
Correo electrónico	<code>edwin.munoz2@uexternado.edu.co</code>

Presentación del curso

La asignatura **Métodos Numéricos** introduce los fundamentos de la **computación científica**, proporcionando las herramientas necesarias para comprender cómo los computadores representan los números, controlan los errores de aproximación y resuelven problemas matemáticos mediante algoritmos numéricos.

El curso integra teoría y práctica utilizando **Python**, haciendo énfasis en tres aspectos fundamentales:

- ▶ Precisión en los cálculos numéricos.
- ▶ Estabilidad de los algoritmos.
- ▶ Eficiencia computacional.

Propósito

Comprender los principios matemáticos y computacionales que permiten resolver problemas mediante métodos numéricos de forma confiable y eficiente.

Temas principales del curso

- ▶ Representación numérica.
- ▶ Análisis del error.
- ▶ Cálculo numérico.
- ▶ Álgebra lineal numérica.
- ▶ Ecuaciones no lineales.
- ▶ Interpolación.
- ▶ Simulación Monte Carlo.
- ▶ Introducción a la optimización numérica.

¿Por qué es importante este curso?

Los conceptos desarrollados constituyen la base matemática y computacional para asignaturas como **Optimización**, **Machine Learning** y **Deep Learning**.

Competencias y resultados de aprendizaje

Competencias

Cognitiva

- ▶ Comprender los fundamentos de los métodos numéricos.
- ▶ Analizar y seleccionar el método más adecuado para resolver un problema.

Comunicativa

- ▶ Comunicar y documentar soluciones con lenguaje matemático y científico.
- ▶ Interpretar y argumentar resultados mediante herramientas computacionales.

Al finalizar el curso, el estudiante será capaz de:

- ▶ Comprender la representación numérica y los errores computacionales.
- ▶ Implementar y comparar métodos numéricos en Python.
- ▶ Evaluar estabilidad, convergencia y costo computacional.
- ▶ Justificar la elección del método más apropiado.
- ▶ Presentar resultados de forma clara y reproducible.

Contenidos del curso: semanas 1 a 8

Semana	Sesión	Tema por día	Bibliografía
1	1	Presentación del curso. Computación científica. Aplicaciones en Ciencia de Datos, Machine Learning y Optimización.	Turner, cap. 1; Nocedal, cap. 1
1	2	Representación de números. Punto flotante IEEE-754.	Turner, cap. 2
2	3	Error absoluto, relativo, redondeo, truncamiento y épsilon.	Turner, cap. 2
2	4	Overflow, underflow y cancelación numérica. Laboratorio en Python.	Turner, cap. 2
3	5	Condicionamiento y propagación del error.	Turner, cap. 2
3	6	Derivación numérica.	Turner, cap. 3
4	7	Integración numérica.	Turner, cap. 3
4	8	Aplicaciones del cálculo numérico.	Turner, cap. 3
5	9	Taller integrador. Primera fase del Proyecto Integrador.	Turner
5	10	Parcial I.	—
6	11	Eliminación de Gauss y pivoteo.	Turner, cap. 4
6	12	Factorización LU.	Turner, cap. 4
7	13	Métodos de Jacobi y Gauss-Seidel.	Turner, cap. 4
7	14	Mínimos cuadrados y regresión.	Turner, cap. 4
8	15	Valores propios y PCA.	Turner, cap. 4
8	16	Laboratorio con NumPy.	Turner, cap. 4

Contenidos del curso: semanas 9 a 16

Semana	Sesión	Tema por día	Bibliografía
9	17	Bisección, punto fijo, Newton y secante.	Turner, cap. 5
9	18	Método de Newton para sistemas.	Turner, cap. 5
10	19	Taller integrador. Segunda fase del Proyecto Integrador.	Turner
10	20	Parcial II.	—
11	21	Interpolación de Lagrange y Newton.	Turner, cap. 6
11	22	Splines.	Turner, cap. 6
12	23	Descenso del gradiente y tasa de aprendizaje.	Nocedal, caps. 1–2
12	24	Momentum, AdaGrad, RMSProp y Adam.	Goodfellow et al. (2016), cap. 8; Ruder (2016)
13	25	Generación de números pseudoaleatorios.	Kroese, cap. 1
13	26	Simulación de variables aleatorias.	Kroese, cap. 3
14	27	Simulación Monte Carlo.	Kroese, cap. 3
14	28	Introducción a la optimización numérica.	Nocedal, cap. 1
15	29	Taller integrador.	Turner; Kroese; Nocedal
15	30	Parcial III.	—
16	31	Proyecto final.	Material del curso
16	32	Retroalimentación y articulación curricular.	Material del curso

Metodología del curso

¿Cómo trabajaremos?

- ▶ Clases magistrales.
- ▶ Ejemplos y demostraciones.
- ▶ Laboratorios en Python.
- ▶ Talleres individuales y grupales.
- ▶ Proyecto Integrador.

Recursos

- ▶ Cuadernos de Python (Jupyter).
- ▶ Aula virtual.
- ▶ Material bibliográfico.
- ▶ Repositorio del curso.

Énfasis del curso

- ▶ Comprender antes que memorizar.
- ▶ Experimentar con algoritmos numéricos.
- ▶ Analizar precisión, estabilidad y convergencia.
- ▶ Comparar métodos y justificar su elección.
- ▶ Resolver problemas de Ciencia de Datos.

Ruta de aprendizaje

Teoría → Implementación → Análisis → Aplicación

Evaluación del curso

Instrumentos de evaluación

- ▶ Ejercicios en clase.
- ▶ Quices.
- ▶ Talleres.
- ▶ Parciales.
- ▶ Proyecto Integrador.

Enfoque

La evaluación será **continua, formativa y sumativa**, privilegiando el aprendizaje, la retroalimentación y el desarrollo progresivo de competencias.

Distribución de la nota

Actividad	Semana	%
Parcial I	5	20
Parcial II	10	20
Parcial III	15	20
Trabajo en clase	Permanente	20
Proyecto Integrador	5, 10 y 16	20

Proyecto de seguimiento al aprendizaje mediante audios

¿En qué consiste?

Cada semana, los estudiantes grabarán un audio de aproximadamente **1 minuto** respondiendo:

- ▶ ¿Qué aprendí esta semana?
- ▶ ¿Qué dificultades tuve?
- ▶ ¿Qué dudas aún tengo?

Objetivo

Fortalecer el proceso de aprendizaje mediante herramientas de analítica de datos e inteligencia artificial, generando alertas tempranas sobre posibles dificultades de comprensión.

Evaluación

El envío oportuno del audio hará parte de la nota de **Quices (información por confirmar)**:

Viernes: 5.0

Sáb.-Lun.: 3.0

Después: 1.0

¿Por qué estudiar computación científica?

Problemas reales

Muchos problemas actuales en salud, finanzas, recursos naturales y otras disciplinas científicas requieren integrar:

- ▶ Programación.
- ▶ Matemáticas.
- ▶ Métodos numéricos.
- ▶ Modelación matemática.
- ▶ Simulación.
- ▶ Análisis de datos.

Idea central

La computación científica permite formular modelos matemáticos y construir soluciones computacionales para estudiar problemas del mundo real.

Fuente: Turner (2018), cap. 1.

Del problema real al modelo matemático

Un modelo matemático

Es una representación de un sistema o situación que permite:

- Comprender un problema real.
- Obtener información cualitativa o cuantitativa.
- Predecir comportamientos futuros.
- Apoyar la toma de decisiones.

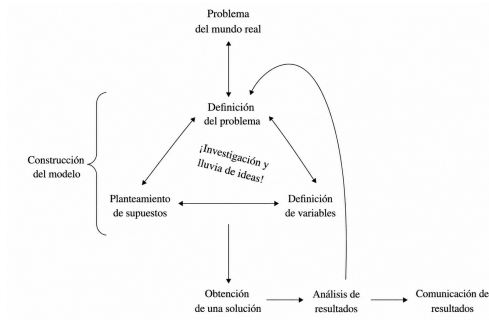


Figura: Proceso de modelación matemática. Adaptado de Turner, Arildsen y Kavanagh (2018, Fig. 1.1).

Fuente: Turner (2018), Fig. 1.1.

Proceso de modelación matemática

1. **Definir el problema:** establecer claramente lo que se desea obtener.
2. **Formular supuestos:** simplificar el problema e identificar los factores relevantes.
3. **Definir variables:** distinguir variables independientes, dependientes y parámetros.
4. **Obtener una solución:** evaluar funciones, ejecutar simulaciones o aproximar numéricamente.
5. **Analizar el modelo:** verificar si los resultados tienen sentido e identificar posibles mejoras.
6. **Comunicar los resultados:** explicar cómo debe utilizarse el modelo.

Característica del proceso

La modelación es iterativa: sus componentes pueden revisarse varias veces y no necesariamente se desarrollan en un orden estrictamente lineal.

Computación científica aplicada

Proceso general

1. Plantear un problema real.
2. Construir un modelo matemático.
3. Obtener una solución computacional.
4. Analizar los resultados.
5. Refinar el modelo.

Selección del método

Una vez formulado el problema matemático, debe seleccionarse la herramienta numérica adecuada para obtener una solución.

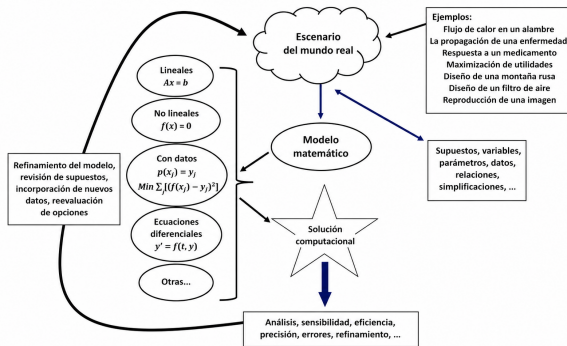


Figura: Ejemplos de problemas resultantes de la modelación matemática de problemas del mundo real. Adaptado de Turner, Arildsen y Kavanagh (2018, Fig. 1.2).

¿Qué problemas pueden estudiarse?

Modelos y métodos

- ▶ Sistemas de ecuaciones lineales.
- ▶ Ecuaciones no lineales.
- ▶ Ecuaciones diferenciales.
- ▶ Interpolación.
- ▶ Integración numérica.

Aplicaciones

- ▶ Ajuste de datos mediante mínimos cuadrados.
- ▶ Simulación y números aleatorios.
- ▶ Optimización.
- ▶ Análisis de sensibilidad.
- ▶ Evaluación de precisión y eficiencia.

Conexión con Ciencia de Datos y ML

En el capítulo aparecen elementos directamente relacionados con el análisis de datos: ajuste de modelos, mínimos cuadrados, simulación, optimización y uso de herramientas computacionales.

Fuente: Turner (2018), secciones 1.1 y 1.2.

Python como herramienta de trabajo

Características señaladas por Turner

Python es un lenguaje:

- ▶ De código abierto.
- ▶ Interpretado.
- ▶ Multiplataforma.
- ▶ Flexible y fácil de comenzar a utilizar.
- ▶ Con bibliotecas para matemáticas, Ciencia de Datos, modelación, visualización y diseño de videojuegos.

NumPy

Cómputo numérico basado en vectores y matrices.

SciPy

Álgebra lineal, integración, optimización y otros métodos científicos.

Matplotlib

Visualización de datos numéricos y resultados.

Fuente: Turner (2018), sección 1.3.

Bibliotecas fundamentales en Python

Para utilizar las bibliotecas científicas, Turner propone importarlas de la siguiente manera:

Importación

```
import numpy as np
import scipy as sp
import matplotlib.pyplot as plt
```

- ▶ np representa a NumPy.
- ▶ sp representa a SciPy.
- ▶ plt representa el módulo de visualización de Matplotlib.

Fuente: Turner (2018), sección 1.3.

Las soluciones computacionales son aproximadas

Fuentes de error

Las soluciones computacionales pueden estar afectadas por:

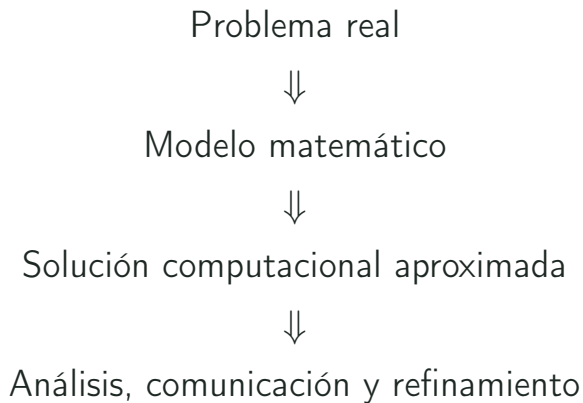
- ▶ Errores humanos o de programación.
- ▶ Errores de truncamiento.
- ▶ Errores de redondeo.
- ▶ Errores asociados con la representación finita de los números.
- ▶ Errores de modelación.

Evaluación del modelo

Una solución debe examinarse considerando:

- ▶ Los supuestos realizados.
- ▶ La precisión de los resultados.
- ▶ La correspondencia entre el modelo y el problema real.
- ▶ La necesidad de refinar el modelo.

Idea central de la computación científica



Conclusión

La computación científica combina modelación matemática, métodos numéricos y programación para estudiar problemas reales cuya solución suele ser computacional y aproximada.

¿Por qué es importante la optimización?

La optimización en el mundo real

La optimización constituye una herramienta fundamental para apoyar la toma de decisiones y analizar sistemas complejos. Sus aplicaciones incluyen, entre otras:

- ▶ Selección de portafolios de inversión.
- ▶ Diseño y operación eficiente de procesos industriales.
- ▶ Diseño óptimo en ingeniería.
- ▶ Análisis de sistemas físicos que evolucionan hacia estados de energía mínima.
- ▶ Determinación de trayectorias de mínimo tiempo en fenómenos naturales.

Importante

Una vez construido un modelo matemático, la optimización permite encontrar la mejor solución posible de acuerdo con un objetivo y un conjunto de restricciones.

Fuente: adaptado de Nocedal y Wright (2006), Capítulo 1.

¿Por qué estudiar Métodos Numéricos en Ciencia de Datos?

Datos $\implies$ Modelo $\implies$ Algoritmos Numéricos $\implies$ Decisiones

Durante este curso aprenderás a:

- ▶ Comprender cómo los computadores realizan cálculos.
- ▶ Analizar la precisión y los errores numéricos.
- ▶ Implementar algoritmos científicos en Python.
- ▶ Resolver problemas mediante métodos numéricos.

Estas bases son fundamentales para:

- ▶ Optimización.
- ▶ Machine Learning.
- ▶ Deep Learning.
- ▶ Simulación Monte Carlo.
- ▶ Modelación matemática.

Los modelos inteligentes solo son tan buenos como los algoritmos numéricos que los hacen posibles.

Bibliografía

-  Turner, P. R. (2018). *Applied Scientific Computing with Python*. Springer.
-  Nocedal, J., & Wright, S. J. (2006). *Numerical Optimization*. Springer.
-  Kroese, D. P., Taimre, T., & Botev, Z. I. (2011). *Handbook of Monte Carlo Methods*. Wiley.
-  Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.